Hard-Coding rag.py in Python

Retrieve top-k chunks, format them, ground the prompt, and return a string answer.

Project: logistics-rag-agent

File: rag.py

Pipeline: question -> Chroma -> prompt -> Ollama chat -> str

rag.py is the Retrieval-Augmented Generation path WITHOUT tools. It always searches documents, stuffs chunks into GROUNDED_PROMPT, and asks the LLM.

1. Big picture

```
question
   |
   +--> retriever (top-k similar chunks from chroma_db)
   |         |
   |         v
   |    log_retrieved_chunks -> format_docs -> {context}
   |
   +--> {question} (passthrough)
   |
   v
GROUNDED_PROMPT -> ChatOllama -> StrOutputParser -> answer str
```

WHY

This is LCEL (LangChain Expression Language): pipe objects with |. Each stage is a Runnable you can test alone.

2. Build step by step

Step 1: Imports and shared logger name

```
from __future__ import annotations

import logging
from pathlib import Path
from typing import Any

from langchain_core.documents import Document
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable, RunnableLambda, RunnablePassthrough
from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_chroma import Chroma

from prompts import GROUNDED_PROMPT

logger = logging.getLogger("logistics_rag")
```

- from __future__ import annotations -- postpones evaluation of type hints (cleaner forward refs)

- RunnableLambda -- wrap a plain Python function so it can sit in a | pipeline

- RunnablePassthrough -- copy the input question into the 'question' field unchanged

- Same logger name "logistics_rag" as tools/agent -- one -v flag controls all debug logs

WHY

Reusing get_vectorstore settings here (and in ingest) keeps query embeddings in the SAME vector space as ingested chunks. Mismatched EMBED_MODEL => empty/meaningless search.

Step 2: Hard-code Chroma + model constants

```
CHROMA_DIR = Path(__file__).parent / "chroma_db"
COLLECTION = "logistics_docs"
EMBED_MODEL = "nomic-embed-text"
CHAT_MODEL = "llama3.1:8b"
TOP_K = 4
```

- CHROMA_DIR / COLLECTION / EMBED_MODEL -- must match ingest.py

- CHAT_MODEL -- local Ollama chat model for generation

- TOP_K = 4 -- retrieve 4 nearest chunks (tune: too few miss facts, too many add noise)

WHY

TOP_K is a retrieval hyperparameter, not magic. Start around 3-8 for SOP PDFs.

Step 3: Open the existing vector store

```
def get_vectorstore() -> Chroma:
    return Chroma(
        persist_directory=str(CHROMA_DIR),
        embedding_function=OllamaEmbeddings(model=EMBED_MODEL),
        collection_name=COLLECTION,
    )
```

Same helper pattern as ingest.py, but rag.py only READS. It does not add documents.

Step 4: Format Document list into prompt-ready text

```
def format_docs(docs: list[Document]) -> str:
    if not docs:
        return "(No relevant context retrieved.)"

    parts: list[str] = []
    for i, doc in enumerate(docs, start=1):
        name = doc.metadata.get("filename") or Path(
            doc.metadata.get("source", "unknown")
        ).name
        page = doc.metadata.get("page")
        page_note = f", page {page}" if page is not None else ""
        parts.append(f"[{i}] ({name}{page_note})\n{doc.page_content.strip()}")
    return "\n\n".join(parts)
```

- Empty list => explicit placeholder so the model sees 'no context' clearly

- filename fallback to Path(source).name -- works even if filename meta missing

- Numbered [1], [2] blocks -- easier for the model (and you) to cite

WHY

The LLM never sees raw Document objects. format_docs is the bridge from retrieval objects to the {context} string slot in prompts.py.

Step 5: Log chunks for debugging, then pass through

```
def log_retrieved_chunks(docs: list[Document]) -> list[Document]:
    """Log retrieved chunks for debugging, then pass them through."""
    ...
    return docs  # IMPORTANT: same list continues down the pipe
```

Side-effect function: prints previews at DEBUG level when you run app.py -v. It MUST return docs unchanged so the next stage still receives the list.

WHY

When answers look wrong, the bug is often retrieval (wrong chunks), not the LLM. Logging chunks first saves hours.

Step 6: Build the LCEL chain

```
def build_rag_chain(
    *,
    k: int = TOP_K,
    model: str = CHAT_MODEL,
    temperature: float = 0.0,
) -> Runnable[Any, str]:
    vectorstore = get_vectorstore()
    retriever = vectorstore.as_retriever(search_kwargs={"k": k})
    llm = ChatOllama(model=model, temperature=temperature)

    return (
        {
            "context": (
                retriever
                | RunnableLambda(log_retrieved_chunks)
                | RunnableLambda(format_docs)
            ),
            "question": RunnablePassthrough(),
        }
        | GROUNDED_PROMPT
        | llm
```

```
        | StrOutputParser()
    )
```

Signature details

- * -- forces k/model/temperature to be keyword-only (build_rag_chain(k=5))
- temperature=0.0 -- more deterministic, less creative invention
- Return type Runnable[Any, str] -- invoke with a question str, get answer str

Dict branch meaning

- Left side keys become prompt variables
- retriever runs on the input question automatically
- RunnablePassthrough copies the same input into 'question'

WHY

as_retriever(search_kwargs={'k': k}) turns Chroma into a LangChain Retriever with a stable .invoke(query) -> list[Document] interface.

Step 7: One-shot convenience wrapper

```
def ask(question: str, *, k: int = TOP_K) -> str:
    """Run one grounded Q&A turn."""
    chain = build_rag_chain(k=k)
    return chain.invoke(question)
```

WHY

app.py and tests call ask() without caring about LCEL plumbing. Rebuilds the chain each call (simple; fine for a CLI demo).

3. Practice

- Run ask('What is the cold chain rule?') after ingest
- Set TOP_K=1 vs 8 and compare answer quality
- Enable -v in app.py and read logged chunk previews

4. Common failures

- Empty/irrelevant answers -- chroma_db missing or wrong EMBED_MODEL
- Ollama errors -- pull llama3.1:8b and nomic-embed-text
- Hallucinations -- prompt rules too weak or temperature > 0